Phoenix Project - Backend Architecture & Logic Explanation (Hinglish)

Ye document Phoenix project ke backend (server aur database connection) ki working ko samjhane ke liye banaya gaya hai. Aap iska use teachers ko backend flow asani se samjhane ke liye kar sakte hain.

1. Technology Stack (Upyog ki gayi technologies)

- **Node.js & Express.js**: HTTP Server banane aur RESTful APIs ko host karne ke liye ye web framework use kiya gaya hai.
- **Supabase (PostgreSQL Database)**: System ka entire data (Cars inventory, Users profiles, Reservations) store karne aur Database queries (CRUD operations) ke liye use hua hai.
- **CORS & Dotenv**: *CORS* (Cross-Origin Resource Sharing) React frontend aur Express backend ko secure tareeqe se connect karta hai. *Dotenv* server ke API keys (secrets) ko safe rakhne mein madad karta hai.

2. Server Setup & Entry Point (`server.js`)

- `server.js` hamare backend ka starting point (main file) hai.
- Jab server start hota hai, toh sabse pehle middleware (`express.json()` aur `cors()`) active hote hain taaki frontend se JSON format mein data bheja aur padha ja sake.
- Uske baad request URLs ko unki route files se connect kiya jata hai:
    - `/api/auth` -> Authentication & user profiles ke routes.
    - `/api/cars` -> Gaadiyon ki listing, create aur update karne ke routes.
    - `/api/reserved` -> Car booking aur reservation status handle karne ke routes.
    - `/api/admin` -> Dashboard graphs aur admin tools ke liye.

3. Modular Architecture (Folder Structure)

Backend code ko maintainable (clean) rakhne ke liye 'Controller-Service Pattern' ka use kiya gaya hai:

A. Routes Layer (`/routes` folder)

- **Kaam**: Inka kaam sirf frontend ki HTTP Request (`req`) ko receive karna aur response (`res`) bhej dena hai. Yahan actual database language nahi likhi jati.
- Jaise jab frontend `GET /api/cars` par req bhejta hai, toh `cars.js` receive karta hai, error handling try-catch lagata hai, aur asal kaam Service Layer se karwata hai.

B. Services Layer (`/services` folder)

- **Kaam**: Database operations aur Asli (Core) Business Logic yahin likha gaya hai.
- **`carService.js`**: Frontend se aane wale filters (jaise brand, price, search) service pakadti hai, aur query banati hai `supabase.from('cars').select('*')` aur pagination ke sath actual database search perform karke result routes ko lautati hai.
- **`reservationService.js`**: `bookCar` (gaadi book karna) ka calculation aur database record insertion isi layer mein completely isolated tareeqe se run hota hai.

4. Security & Authentication (`/middleware/auth.js`)

- Backend par APIs ko direct open/unsecure nahi chhoda gaya hai kyu ki koi bhi gadiyan book karne wali api directly invoke (call) kar sakta tha. Iske liye `authenticateToken` middleware ka sahara liya gaya hai.
- **Working Flow**:
    1. Frontend HTTP request ke 'headers' mein JWT (Bearer) token bhejta hai.
    2. Middleware us token ko nikalta hai aur sidhe Supabase ke secure server par `supabase.auth.getUser(token)` match karne bhejta hai ki "kya ye asli user hai ya hacker hai?".
    3. Agar verified nikalta hai toh humaari Database ki `profiles` table ko double check karke dekhta hai ki user ban toh nahi ho gaya ya isActive hai.
    4. Finally request verify karke permission milti hai. Jo api sirf boss/admin use kar skta hai usme extra `requireAdmin` check function laga diya jata hai.

---

5. Detailed API Endpoints Flow (Routes / 'Pages' ki working)

Backend par 'pages' ka matlab APIs (Endpoints) hota hai jinke basis par frontend ki alag-alag screen render (chalti) hoti hai. Niche un endpoints ki deeply working explain ki hai:

A. Auth Routes (`/routes/auth.js`)

- **Login/Register (`/login`, `/register`)**: Inka logic specially frontend based rakha gaya hai (jaise authentication frontend se direct Supabase tak handle ho), par verification handle karne ke liye API support deti hai.
- **Auth Callback (`/callback`)**: Jab user successful register karta hai tab token verify karke Database me User Profile (`profiles` table) create / sync hoti hai.
- **Profile (`/profile`)**: Update aur Get methods provide karta hai jisse user apna naam, number aur avatar (dp) update kar sakta hai. Is route me `authenticateToken` strict rehta hai.

B. Cars Inventory Routes (`/routes/cars.js`)

- **GET `/` (Cars List)**: Saari cars search parameters (jaise budget aur brand limit) limit, sort aur page offset ke sath backend par laye jate hain aur Service Layer unhe search banakar frontend array bhej deti hai.
- **GET `/featured/all`**: Ye endpoint specifically Home.js ko pehli look feature karne ke liye latest gadi bhejta hai.
- **POST/PUT/DELETE (`/:id`)**: Ye admin-secured endpoints hain, jahan par `requireAdmin` filter pas kiye bina koi gaadi upload, delete ya update (price change) nahi kar sakta.

C. Booking/Reservation Routes (`/routes/reservations.js`)

- **POST `/book`**: User yahan ID pass karta hai (`carId`). Backend user ki details verification me extract karta hai aur reservation entry create kar deta hai.
- **GET `/my-reservations`**: Ye strict filter route hai. Normal user API layega to wahi data aayega jis table column me uska id `user_id` record ho. Usse koi aur aadmi dusre ka ticket nahi check kar sakta.
- **Status Change (`/:id/status`)**: Booking ki state ('pending' se 'delivered' tak) change karne ka pure backend logic yahi resolve aur secure hota hai.

D. Admin Dashboard Routes (`/routes/admin.js`)

- **Dashboard Stats (`/stats`)**: Ye route single call me kai parallel Queries (`Promise.all()` ka use karke) fire karta hai:
    1. *Total Sales, Tickets*: reservations search kar revenue value count karta hai.
    2. *Revenue Trend Graph*: Khali Database hits bhej kar array list nahi laata, pichhle 30 din ki sales array object format me arrange karta hai, Map objects aur Timestamp logic convert karta hai taaki frontend perfectly us data value ko Graph (SVG) mein render kar sake without lag.
- **User Manage (`/users/:id`)**: System me admin kisi ko bhi view, ban aur update kar sakta hai yaha se. Block feature isi user row object ko edit karke work karti hai.

Conclusion/Summary (Teachers ke liye):

"Sir/Mam, hamara Express.js Backend pure API-driven aur modular 'Service Architecture' based hai. Humne application logic (services) ko routing logic se ekdum alag rakha hai taaki code scalable rahe. Database ke taur par Supabase ki PostgreSQL DB directly `@supabase/supabase-js` SDK se server par fetch ho rahi hai aur security maintain karne ke liye frontend ke un-authenticated HTTP requests middleware levels (JWT token checking aur Admin role check) cross karke hi data tak pohoch sakte hain."